
Chapter 5 | 整机集成与端到端推理

- 本章目标：Hack CPU + NPU 组成完整系统。
- 内容：MMIO 扩展、Hack 驱动、集成测试。
- 最终成果：一条命令完成 MNIST 分类。

Chapter 5 | Hack + NPU 系统

- CPU：运行 Hack 机器码，负责调度。
- Memory：RAM/Screen/Keyboard + NPU MMIO。
- NPU：独立时钟域或同源时钟，完成计算。
- 数据流：ROM 程序 -> CPU -> MMIO -> NPU -> RAM。

Chapter 5 | MMIO 地址分配

- 0x0000-0x3FFF : RAM16K。
- 0x4000-0x5FFF : Screen。
- 0x6000 : Keyboard。
- 0x7000-0x7FFF : NPU 寄存器与缓冲区（新增）。

Chapter 5 | 命令协议

- CPU 把图片地址、权重地址、层数写入寄存器。
- CPU 写 CMD=START, NPU 开始执行。
- NPU 执行时置 BUSY=1。
- 完成后 BUSY=0、DONE=1, 结果写入约定地址。

Chapter 5 | Hack 驱动

- 汇编伪代码：写 ADDR_SRC、写 ADDR_DST、写 CMD、轮询 STATUS。
- Hack 没有函数调用栈，用小标签和跳转即可。
- 驱动只做 I/O 和地址计算，不做矩阵运算。
- 完成后读取结果 RAM，比较 argmax。

Chapter 5 | 轮询 vs 中断

- 轮询：CPU 循环读 STATUS，简单可靠。
- 中断：需要扩展 Hack 指令集或外设引脚，复杂。
- 本章用轮询，符合 nand2tetris 的极简风格。
- 真实 SoC 中轮询对小任务完全够用。

Chapter 5 | 集成测试

- 用 Ch4 的一张测试图片作为输入。
- NPU 权重在仿真启动时从文件加载。
- CPU 跑驱动，NPU 跑推理。
- testbench 检查最终类别并打印 PASS。

Chapter 5 | 调试方法

- 先单独测 MMIO 读写，再测 NPU 状态机。
- 用 VCD 波形检查 CMD/STATUS 握手。
- 逐层比对 NPU 输出与 golden。
- 不一致时从最后一层往前定位。

Chapter 5 | 性能度量

- 统计从 START 到 DONE 的周期数。
- 对比理论 MAC 周期数，算出阵列利用率。
- 瓶颈通常在 DMA/权重加载，而不是阵列。
- 利用率报告是本章加分项。

Chapter 5 | 扩展方向

- 把 MNIST 换成 CIFAR-10 或自定义数据集。
- 增加批量推理与流水。
- 评估 tiny YOLO 的 conv 部分。
- NMS 等后处理可以继续留在 CPU/Python。

Chapter 5 | 本章任务

- 扩展 Memory/Computer 的 MMIO。
- 写 Hack 驱动并编译。
- 跑通端到端 MNIST 推理。
- 保持全部历史测试通过。

Chapter 5 | 总结

- 从 Nand 到 Hack，再到 NPU：一条完整的现代计算机链路。
- MMIO 是 CPU 与外设之间的桥梁。
- 小分类模型是端到端验证的完美载体。
- 后续可以按同样框架挑战 tiny YOLO。